

Assignment

Neurosymbolic AI – Semantic Loss

1 Training a Neural Classifier

The standard way of training a neural network that performs classification is to pass subsymbolic input through a neural network and use a softmax in the output layer. This ensures that the neural network encodes a proper conditional probability distribution.

More formally, let f denote the neural network and let x be the input that we wish to classify, The neural network f where the last layer is a softmax encodes a proper probability distribution

$$p(k | x) = f_k(x). \quad (1)$$

with $f(x) = (f_1(x), \dots, f_K(x))$ being a vector valued function. The entries of the output vector correspond to the events of categorical probability distribution:

$$\sum_{k=1}^K f_k(x) = 1 \quad (2)$$

$$\forall k \in \{1, \dots, K\} : f_k(x) \geq 0 \quad (3)$$

Q1 Build a simple MNIST classifier that uses a softmax in the final layer and use the cross-entropy as loss. Evaluate your classifier on a test set and report the results.

There are plenty of MNIST classifiers available online. No need to reinvent the wheel! Take something small that works well enough. We will use this classifier as a baseline. Be sure, however, that you report the source. It is also preferable that you use PyTorch to implement the classifier as for **Q3** you will be given code using PyTorch already.

2 Replacing Softmax with Sigmoids

The softmax function $\text{sigma}(x) \in \mathbb{R}_{>0}^K$ in the last layer of the neural network, which has the following functional form

$$\sigma_i(x) = \frac{\exp(x_i)}{\sum_{j=1}^K \exp(x_j)} \quad (4)$$

ensures that we have a properly normalized distribution. We will now replace the single softmax with multiple sigmoids $s(x_i) \in \mathbb{R}_{>0}$:

$$s(x_i) = \frac{1}{1 + \exp(-x_i)} \quad (5)$$

Q2.1 Repeat the experiment from **Q1** but replace the single softmax with multiple sigmoid. Report the results and contrast them to the results from **Q1**. You should observe a dramatic decrease in accuracy. Note: do not change the loss function to binary cross entropy but keep cross-entropy.

Q2.2 Use again sigmoids and now change to a binary cross-entropy loss. Explain what is going on.

3 Encouraging Logical Constraints with the Semantic Loss

We are now going to learn about the semantic loss and how to encourage constraints onto the outputs of our neural network. Start by going through the paper that introduces the semantic loss (Xu et al., 2018). Section 5 is the most important one to understand for this assignment.

In order to implement the semantic loss we are going to use the PySDD library¹.

Q3.1 Write a Python function that implements the exactly-one constraint using the PySDD library. Your function should return a PySDD formula. Report the Python code.

Q3.2 What does the exactly-one constraint have to do with the softmax function?

We can give the PySDD library a Boolean function and it returns a so-called arithmetic circuit (also described in the semantic loss paper). This arithmetic circuit encodes the constraint described by the Boolean formula in an end-to-end differentiable way.

Q3.3 In the accompanying code you can find already an implementation for a depth first circuit evaluation function. However, only the probability semiring is used. This is problematic as using probabilities often leads to numerical stability issues. Finish the implementation of the LogProbSemiring class in the `evaluator.py` file and test your code using the `main.py` file. You will have to uncomment certain lines. In the `evaluator.py` you will find the commented-out `log1mexp` function. It might come in handy.

Q3.4 Implement the semantic loss and repeat the experiment from **Q2**. Report the results and contrast them to the results from **Q2**. Give a quantitative and a qualitative analysis.

4 Open-Ended Question

Q4 Come up with your own neurosymbolic problem where you would like to encourage a symbolic constraint on the outputs of a neural network.

Hint: Our naive implementation does not scale all that well. For larger logical constraints with need to be a bit more sophisticated and use for example the methods introduced by (Maene, Derkinderen, and Zuidberg Dos Martires, 2025) (<https://github.com/ML-KULeuven/klay>).

5 Neurosymbolic AI and Physics Informed Neural Networks

Q5 Read up on *physics informed neural networks* (Raissi, Perdikaris, and Karniadakis, 2019) and compare them to the semantic loss. What are differences?
<https://www.youtube.com/watch?v=-zrY7P2dVC4>

References

- Maene, Jaron, Vincent Derkinderen, and Pedro Zuidberg Dos Martires (2025). “KLayer: Accelerating Arithmetic Circuits for Neurosymbolic AI”. In: *Proceedings of the International Conference on Learning Representations*.
- Raissi, Maziar, Paris Perdikaris, and George E Karniadakis (2019). “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. In: *Journal of Computational physics* 378, pp. 686–707.
- Xu, Jingyi et al. (2018). “A semantic loss function for deep learning with symbolic knowledge”. In: *International conference on machine learning*. PMLR, pp. 5502–5511.

¹<https://github.com/ML-KULeuven/PySDD>